

Hat Customizer System

Problem

Hats on usernames are manually created by Sean for each supporter. This doesn't scale and users can't customize their own hats.

Solution

A hat customizer tool where paying supporters upload their own hat image, position it on a live preview of their username, and save. Coolhole renders hats on usernames in both the userlist and chat messages. Stripe handles all billing. Coolhole stores zero customer PII.

User Flow

1. User clicks "Get a Hat" on Coolhole
2. Redirected to Stripe Checkout (hosted on Stripe's domain — card info never touches Coolhole)
3. Stripe charges \$5/month, fires webhook to Coolhole
4. Coolhole activates hat access for that username
5. User visits /hat s customizer page
6. Upload a hat image (PNG with transparency, max 256x256)
7. Live preview shows their Coolhole username with the hat overlaid
8. Drag to position, resize, rotate
9. Save — hat appears on their name in chat and userlist immediately
10. Can change/remove hat anytime while subscription is active
11. If subscription lapses — hat stops rendering, config preserved for reactivation

Security Model

Coolhole stores NO customer PII. No emails, no card numbers, no billing addresses.

What Coolhole stores:

- Coolhole username (already public in chat)
- Stripe customer/subscription IDs (opaque strings, useless without the Stripe secret key)
- Hat image and position config
- Active/inactive boolean

What Stripe holds (not us):

- Credit card details
- Email addresses
- Billing history

- Payment retry/dunning logic
- Renewal reminder emails (built-in toggle in Stripe dashboard)
- PCI Level 1 compliance

If the **Coolhole DB leaked**, an attacker gets: usernames, hat positions, and Stripe IDs that are meaningless without the API secret key. No actionable customer data.

What we DO secure:

- `sk_live...` Stripe secret key — in `config.yaml` only, never in code or git
- `whsec_...` webhook signing secret — verify every webhook is actually from Stripe
- Hat upload endpoint — validate file type server-side, size limit, strip EXIF, re-encode through sharp
- Admin routes — gated behind CyTube rank ≥ 255 (siteadmin)
- HTTPS required (especially for webhook endpoint)

Payment Flow (Stripe)

Checkout

1. User clicks "Get a Hat" → server creates a Stripe Checkout Session with username in metadata
2. User redirected to `checkout.stripe.com` — enters payment there (never on Coolhole)
3. On success, Stripe redirects back to Coolhole `/hats` page

Webhooks (POST `/stripe/webhook`)

Listen for these events:

- `checkout.session.completed` — activate hat, store `stripecustomerid` and `stripesubscriptionid`
- `customer.subscription.deleted` — set `active = false`, hat stops rendering
- `invoice.payment_failed` — set `active = false` after Stripe exhausts retries
- `customer.subscription.updated` — handle plan changes if we add tiers later

Renewal Reminders

Stripe handles this natively. Enable "Upcoming renewal" emails in Stripe Dashboard → Settings → Billing → Subscriptions. No custom email system needed.

If We Need to Email a User

Look up their email via `stripe.customers.retrieve(stripecustomerid)` on demand. Don't store it.

Data Model

hats table (new, in CyTube MySQL)

```
CREATE TABLE hats (
  username VARCHAR(20) PRIMARY KEY,
```

```

image_path VARCHAR(255),
offset_x INT NOT NULL DEFAULT 0,
offset_y INT NOT NULL DEFAULT -20,
width INT NOT NULL DEFAULT 24,
height INT NOT NULL DEFAULT 24,
rotation INT NOT NULL DEFAULT 0,
stripe_customer_id VARCHAR(64),
stripe_subscription_id VARCHAR(64),
active BOOLEAN NOT NULL DEFAULT FALSE,
admin_note VARCHAR(255),
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

No customer PII in this table. Stripe IDs are opaque references only.

Hat image storage

- Stored on the Coolhole server at `www/hats/<username>.png`
- Served statically at `/hats/<username>.png`
- Images processed on upload: re-encoded through sharp, max 256x256, PNG only, EXIF stripped
- Cache busting: append `?v=timestamp` to hat image URLs

Server Changes

1. `src/database/tables.js` — Add hats table

Add `ensureTable('hats', ...)` following existing pattern.

2. `src/database/hats.js` (new)

```

getHat(username) -> { image_path, offset_x, offset_y, width, height, rotation, active }
setHat(username, config) -> void
removeHat(username) -> void (deletes image, clears config)
getHatsForUsers(usernames[]) -> { [username]: hatConfig } (bulk load for channel join)
getAllHats() -> [{ username, ...config, active, admin_note }] (admin view)
updateHatAdmin(username, changes) -> void (admin can edit position, toggle active, add note)

```

3. `src/web/routes/hats.js` (new)

- GET `/hats` — serves the customizer page (requires login + active subscription)
- POST `/hats/upload` — receives hat image, validates, processes through sharp, stores
- POST `/hats/save` — saves position/size/rotation config
- DELETE `/hats` — removes user's hat image and config
- POST `/hats/checkout` — creates Stripe Checkout Session, returns redirect URL

4. `src/web/routes/stripe.js` (new)

- POST `/stripe/webhook` — receives Stripe events, verifies signature with `whsec_...`

- Handles: `checkout.session.completed`, `customer.subscription.deleted`, `invoice.payment_failed`, `customer.subscription.updated`
- Flips active flag and stores Stripe IDs

5. `src/web/routes/admin-hats.js` (new)

- GET `/admin/hats` — serves admin panel (rank $\geq$ 255 only)
- GET `/admin/hats/list` — returns all hats with status
- POST `/admin/hats/:username/edit` — edit any user's hat position/size/rotation
- POST `/admin/hats/:username/toggle` — activate/deactivate manually (comp or kill)
- DELETE `/admin/hats/:username` — delete hat image (offensive content)
- POST `/admin/hats/:username/note` — add admin note ("deleted — inappropriate image")

6. `src/channel/channel.js` — Include hat data in user metadata

In `packUserData`, look up hat config for the user. If `active = true`, attach to metadata:

```
meta: {
  afk: ...,
  muted: ...,
  hat: { image_path, offset_x, offset_y, width, height, rotation } // only if active
}
```

Hat data goes on all three visibility tiers (base/mod/sadmin) — it's public info.

7. `src/user.js` — Add hat field

Add hat to user metadata that gets sent to clients via `userList` and `addUser` events.

Client Changes

8. `www/js/util.js` — Render hats on usernames

In `formatChatMessage` (after line 1569):

After `$("#<strong/>").addClass("username")...` creates the username element:

```
if (data.meta.hat) {
  var hat = data.meta.hat;
  var hatImg = $("")
    .attr("src", hat.image_path)
    .css({
      position: "absolute",
      width: hat.width + "px",
      height: hat.height + "px",
      left: hat.offset_x + "px",
      top: hat.offset_y + "px",
      transform: "rotate(" + hat.rotation + "deg)",
      pointerEvents: "none"
    });
}
```

```

name.css("position", "relative").css("overflow", "visible");
hatImg.appendTo(name);
}

```

In addUserToList (after line 3506):

Same pattern — after `var nametag = $("`

```

if (data.meta.hat) {
var hat = data.meta.hat;
var hatImg = $("

```

9. [www/hats/customizer.html](#) (new) — Hat customizer page

The customizer UI:

- Dark theme matching Coolhole
- Canvas area showing the user's username rendered in the same font/color as Coolhole chat
- Upload button for hat image
- Hat overlaid on the username, draggable
- Size slider (8px to 64px)
- Rotation slider (-180 to 180)
- Live preview updates as user drags/adjusts
- Save button
- Preview shows both "in chat" and "in userlist" appearance

10. [www/hats/admin.html](#) (new) — Admin panel

Admin hat management UI:

- Table of all users with hats (username, preview, status, Stripe status, admin note)
- Click any user to open their hat in the same customizer UI (with username selector at top)
- Admin can reposition/resize/rotate any user's hat
- Delete button for hat image (offensive content removal)
- Toggle active/inactive manually (comp someone or kill a hat)

- Add admin notes
- Link to Stripe customer page (opens Stripe dashboard) for billing questions
- Gated behind CyTube siteadmin rank (≥ 255)

Config Additions

config.yaml

```
stripe:
enabled: false
secret-key: '' # sk_live... or sk_test...
publishable-key: '' # pk_live... or pk_test...
webhook-secret: '' # whsec...
price-id: '' # price... (the $5/mo recurring price)
success-url: '/hats?payment=success'
cancel-url: '/hats?payment=cancelled'
hats:
enabled: false
max-image-size: 262144 # 256KB
max-dimensions: 256 # 256x256 px
storage-path: 'www/hats'
```

File Summary

File	Action	Purpose
src/database/tables.js	Modify	Add hats table
src/database/hats.js	New	Hat CRUD + admin queries
src/web/routes/hats.js	New	Hat customizer API + Stripe Checkout session
src/web/routes/stripe.js	New	Webhook handler (signature verified)
src/web/routes/admin-hats.js	New	Admin panel routes
src/channel/channel.js	Modify	Attach hat data to user metadata
src/user.js	Modify	Include hat in user broadcast data
www/js/util.js	Modify	Render hats on usernames (userlist + chat)
www/hats/customizer.html	New	User-facing hat editor
www/hats/admin.html	New	Admin hat management panel
src/config.js	Modify	Add stripe + hats config defaults

config.template.yaml	Modify	Add stripe + hats config section
----------------------	--------	----------------------------------

Dependencies

- Stripe account + API keys
- stripe npm package (webhook verification + Checkout session creation)
- sharp npm package for image processing on upload
- Local Coolhole dev instance for testing

Stripe Setup Checklist

- Create Stripe account (or use existing)
- Create a Product ("Coolhole Hat") and Price (\$5/mo recurring)
- Get API keys (test mode first: `sktest...`, `pktest...`)
- Register webhook endpoint: `https://coolhole.org/stripe/webhook`
- Select events: `checkout.session.completed`, `customer.subscription.deleted`, `invoice.payment_failed`, `customer.subscription.updated`
- Get webhook signing secret (`whsec_...`)
- Enable "Upcoming renewal" emails in Stripe Dashboard → Billing → Subscriptions
- Test full flow with Stripe test cards before going live

Edge Cases

- User changes Coolhole username: hat stays on old name, needs manual migration or username-change hook
- Offensive hat images: admin deletes via admin panel, adds note
- Hat image CDN caching: `?v=timestamp` appended to hat URLs on change
- Stripe webhook replay: use idempotency — check if already processed before toggling state
- Multiple tabs: hat customizer should handle concurrent sessions gracefully
- User subscribes, cancels, resubscribes: hat config preserved, just toggle active

Implementation Order

1. Database table + CRUD module
2. Wire hat data into `packUserData` (server metadata)
3. Client-side hat rendering (`util.js`)
4. API routes (upload, save, delete)
5. Stripe Checkout + webhook handler
6. Customizer UI page
7. Admin panel